# Information at Keyed Improvement Boundaries

Mettapedia Working Notes

31 October 2026

**Abstract**

Five Lean modules characterize sufficient information at a keyed-improvement boundary, relate a source read/remove/add program to an atomic upsert, preserve revision-scoped references, and select between the two programs under explicit primitive costs. The same operation requires different information for different observers: two decision classes, three source-work classes, or $c+2$ result classes for natural candidate $c$. Conditional Kolmogorov bounds connect these finite classes to the project's existing computable universal interpreter. The bounds distinguish retained information from information recoverable through shared context. They do not assert global runtime optimality or native memory safety without an implementation refinement.

## 1 Sufficient information depends on the interface

For a fixed key and natural candidate $c$, let the old slot be either absent or contain $n$. Absence denotes infinity. Improvement returns the new slot and a Boolean indicating whether the strict improvement happened:

$$R_c(o) = \begin{cases} (\text{some } c, \mathsf{true}) & o = \varnothing \text{ or } o = \text{some } n,\ c < n, \\ (\text{some } n, \mathsf{false}) & o = \text{some } n,\ n \le c. \end{cases}$$

An encoding is sufficient for an observation when a decoder recovers that observation from the encoding and the declared shared context. Sufficiency does not permit discarding distinctions that a later admitted observer needs.

| Observation | Shared resources and distinctions | Classes |
|---|---|---|
| Update decision | Fixed candidate; overwrite versus keep | 2 |
| Source primitive debit | Fixed candidate; keep, insert missing, replace existing | 3 |
| New slot and return flag | Fixed candidate; no access to the old slot | $c+2$ |
| Reference resolution | Fixed payload; $n$ distinguishable generations | at least $n$ |

These are bounds on summary alphabets, not requirements to cache the summaries. A component may instead recompute a decision from the live store. That reduces retained metadata but spends work and relies on continued access to the store. The candidate, key, decoder, store and reference namespace are not free global resources merely because an individual theorem conditions on them.

## 2 An exact and wider validation region

Let $b_c(o)$ be the strict-improvement bit. A worker receiving a bit $b$ writes $c$ into the *live* keyed store when $b$ is true, otherwise leaves that store alone, and returns $b$. The Lean theorem

`plan_exact_iff` proves

$$\text{executePlan}(S, k, c, b) = \text{improve}(S, k, c) \quad \Longleftrightarrow \quad b = b_c(S(k)).$$

The reverse implication is the update law. The forward implication follows because the return flag is observable. Consequently this comparison partition is the exact admission region for this worker, not just a conservative guard.

Preparing candidate 7 against old value 10 and publishing against live value 8 is valid. Exact-old-value validation would refuse it. Publishing that same decision against live value 5 is invalid and must decline or recompute. This does not license erasing a stale named row: the worker writes by key using live state, rather than using an old physical pointer.

A three-way source plan additionally preserves whether a successful operation inserts into an empty slot or replaces an occupied slot. The corresponding read/write debits are 2 and 3; a non-improvement costs one read. Thus the changed bit alone cannot recover source work. Conversely every source plan is realized, and every decoder recovering its debit requires at least three codes. The distinction is a theorem, not an estimated compression benefit.

## 3 Exact result classes and conditional descriptions

Without the old slot, a decoder must distinguish every kept value $0, \ldots, c$, plus the successful overwrite case. The executable code is

$$\kappa_c(o) = \begin{cases} n & o = \text{some}\, n,\ n \le c, \\ c+1 & \text{otherwise.} \end{cases}$$

Its type is `Fin(c+2)`. The decoder returns $(\text{some}\, n, \mathsf{false})$ below the final code and $(\text{some}\, c, \mathsf{true})$ at the final code. Lean proves exact decoding, surjectivity through explicit representatives, and

$$\kappa_c(o) = \kappa_c(o') \quad \Longleftrightarrow \quad R_c(o) = R_c(o').$$

Any sufficient finite summary therefore has at least $c + 2$ codes; an encoding into $b$ Boolean coordinates requires $c + 2 \le 2^b$.

The existing natural-word codec encodes each result class as a binary word. For a conditional algorithm $U$, fixed auxiliary context $z$, and reachable class encodings, fewer than $2^L$ classes have plain conditional program complexity below $L$. Instantiating the existing universal conditional interpreter gives

$$\exists i < c + 2 : \qquad C_U(\text{word}(i) \mid z) \ \ge \ \lfloor \log_2(c+2) \rfloor.$$

The context is common to every class in this comparison. It may include $c$; it cannot secretly contain a different answer for each compared result. The separate theorem `output_in_context_constant_cost` proves that $C_U(z \mid z)$ has a uniform constant upper bound. Moving information into shared context changes residual description length, not total retained data. These are plain-complexity statements, not prefix-free chain identities or an executable shortest-program optimizer.

The fixed-candidate qualification is essential. Old values 10 and 12 have the same result code at candidate 7, but candidate 11 separates them. Indeed, Lean proves that the family of all candidate decision queries is injective on old slots. A component promising all those future queries cannot replace its slot by one previously computed comparison bit.

## 4 A source program and its sequential continuations

`ImprovementSourceProgram` uses the existing `AbstractSeparationLogic.Prog` syntax. Its source independently sequences a read, an optional erase of the observed old value, an insert, and a return. Successful operations return the singleton key packet; unsuccessful operations return the empty packet. The target program performs one upsert and returns the same packet. The handler is total and no environment step occurs inside the block.

The direct bounded interpreter proves completion with the same final store, answer packet and semantic source debit. The existing primitive-program cursor theorem transports this result to actual cursor execution. More strongly, `composed_source_native` proves equality after an arbitrary sequential continuation, including its residual state and any fault result, when the primitive traversal allowance is adjusted for the block's changed length. This preserves an enclosing driver's answers without replacing that driver or changing its frontier schedule.

Three different budgets must not be conflated: the traversal fuel that exposes primitive syntax, the semantic debit attributed to the source, and a physical cost model. Source replacement needs three primitive calls, while the target needs one. An additional traversal step exposes each completed return. Pausing after the source erase exposes absence; pausing the target after its upsert does not. Therefore this is a completed-block boundary law, not a permission to fuse across arbitrary primitive interrupts, source budget stops, callbacks or concurrent readers. The lowering from the full MeTTa evaluator and its actual accounting units remains an additional obligation.

## 5 Publication identity and retained readers

`ImprovementPublication` implements an independent generation-bearing slot update. A successful publication increments the selected key's generation and installs the candidate; a non-improvement leaves the state unchanged. The projection to payload values and the Boolean return refines keyed improvement. Unrelated keys, including their reference observations, are unchanged.

Each slot is interpreted through the existing `RevisionedStoreView`. Old live names are rejected after successful replacement, new names resolve, and the original immutable captured view still resolves its old entry. The implementation must keep that captured view's backing storage alive for its readers; logical availability alone does not establish physical retention.

The exact comparison-region law also holds for the full generation-bearing state when publication reads the *live* generation. The cached decision need not copy the generation, while the store must retain enough information to distinguish outstanding reference observations. Even identical payloads do not erase that requirement: $n$ distinguishable generations require at least $n$ metadata codes. Reusing a generation can resurrect an obsolete name. Machine-counter wraparound, reclamation and native linearization require separate proofs; this model uses unbounded naturals.

## 6 Information reduction and profitable execution

`ImprovementPlanSelection` assigns explicit prices to read, erase, put and upsert operations, plus target setup and common admission inspection. It derives exact costs from execution of the source and target programs. The selector's cost is

$$p_{\text{inspect}} + \min\bigl(W_{\text{source}},\ p_{\text{setup}} + p_{\text{upsert}}\bigr).$$

Every selected execution has the same return packet and final store. The selector is optimal among these two modeled choices after inspection, not globally among algorithms. Its cost is at most source work plus inspection. A strict saving against an uninspected source run requires paying inspection as well as setup. Controls show both a profitable replacement and a case where setup defeats the target and inspection still costs work.

The architectural lesson is to minimize residual information relative to a specific observation contract, compose that contract across interfaces, and charge for reconstruction. Neither a smaller representation nor semantic admissibility alone establishes a speed improvement. Different layers can retain different sufficient information without forcing every layer to carry the entire semantic and lifetime context.

## 7 Formal artifacts and remaining refinement

The five modules are under `Mettapedia/Machines/`: `ImprovementInformation`, `ImprovementSourceProgram`, `ImprovementDescriptionLength`, `ImprovementPublication`, and `ImprovementPlanSelection`. `ImprovementInformationAudit` inventories their declaration bodies and transitive primitive axioms. The package reuses the prior keyed-improvement, cursor, revisioned-store, non-factorization and conditional-interpreter developments.

The immediate remaining refinements are the concrete source/profile lowering, failure-aware concurrent native publication with real holder lifetimes, and measured physical prices with amortized conversion/reconstruction costs. The shortest-path driver's observable round counter remains part of the source contract; final-distance equality does not erase it.